CLONESCOPE / SPL-1 DEVELOPMENT JOURNAL

Source-File Input: C++17 Streams, Testing and Git Workflow

Learning Manual 01 | Verified development checkpoint | 25 October 2026

Prepared for SPL-1 development and viva preparation

PROJECT	AUTHOR / SUPERVISOR
CloneScope - An Explainable Code Clone Tagging and Stylometric Analysis Tool	Raian Huq | BSSE1708 3rd Semester, IIT, University of Dhaka
Implementation language: C++17 Development environment: Windows, VS Code, PowerShell, GCC/MSYS2	Supervisor: Dr. Kazi Muheymin-Us-Sakib
Working branch: **feat/source-file-input**	Repository: github.com/raian-x8/SPL_01_BSSE1708

Document purpose
A reproducible explanation of the **implemented and tested** source-file opening and line-reading checkpoint, with exact examples, engineering rationale, test evidence, Git milestones, and defense questions. This is a learning-and-development record, not a plagiarism-detection result.

Implementation status at this checkpoint: opening and printing one hardcoded sample C file **works**. Preprocessing, tokenization, clone detection, stylometry, and configurable folder input are **not yet implemented**.

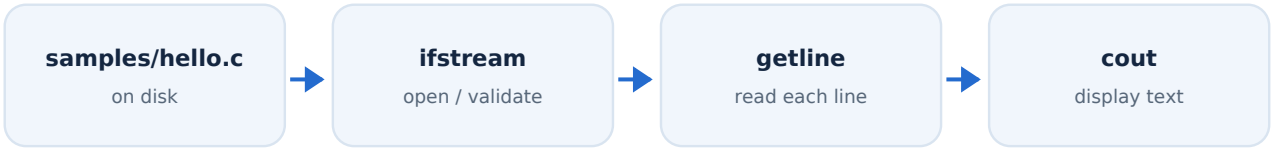
01 / Session record and verified outcomes

The project has two distinct analytical tracks: code-clone tagging and independent stylometric analysis. Both will ultimately provide transparent evidence for instructor review, **not automated plagiarism accusations**. This checkpoint concerns only the beginning of local source-file input.

Milestones completed on 25 October 2026

ORDER	CHECKPOINT	STATUS
01	Merged repository foundation PR #1 into **main**; synchronized local main	Verified
02	Created and published **feat/source-file-input**	Verified
03	Created controlled input **samples/hello.c**	Verified
04	Used **ifstream** and **is_open()** to open a sample source file and handle failure	Built + tested
05	Committed file opening as **e55359f** and published the branch	Verified
06	Implemented **getline()** in a loop to print source lines	Built + tested
07	Tested existing input and nonexistent file; restored sample path	Both passed
08	Staged updated **src/main.cpp** for a separate line-reading commit	Staged; not yet committed at document preparation

What the program does at this checkpoint



Honest scope boundary
The current program **displays** source text; it does not yet store whole-file content in a reusable analysis data structure, discover files automatically, parse functions, count tokens, compare submissions, or generate clone tags.

02 / C++ file-input fundamentals

Headers and names used in this session

SYNTAX	PURPOSE
`#include <iostream>`	Console output: **cout** and **cerr**.
`#include <fstream>`	File streams, including **ifstream** for reading.
`#include <string>`	Provides the **string** type for storing each source line.
`using namespace std;`	Allows unqualified **cout**, **ifstream**, **string**, etc., in this small implementation.

Namespace tradeoff: `std::cout` makes the standard-library origin explicit. `using namespace std;` is legal and reduces repetition in this small file, but it can lead to name collisions. For reusable headers, avoid globally importing the full namespace. The GCC-only `<bits/stdc++.h>` shortcut is intentionally **not used**; standard headers make dependencies clearer and improve portability.

Opening and checking a file

```
ifstream inputFile("samples/hello.c");
if (!inputFile.is_open())
{
    cerr << "Error: Could not open source file.\n";
    return 1;
}
cout << "Source file opened successfully!\n";
```

How it works: `ifstream` constructs an input-file stream and attempts to open a file. `is_open()` reports whether it is associated with an open file. `!` negates that result. `cerr` writes an error message, and returning a nonzero status from `main()` indicates failure.

> **Why relative paths matter**
> `samples/hello.c` is interpreted relative to the **process working directory**, not relative to the location of `main.cpp`. In the verified tests, CloneScope was run from `C:\Users\raian\Developer\SPL_01_BSSE1708`, so the path resolved correctly.

What file opening does not guarantee

A successful open does not mean the file has any content, that the content is valid C, or that subsequent reads cannot fail. The current implementation distinguishes an opening failure, but broader read-error diagnostics remain future work.

03 / Reading every source line

The working loop

```
string line;

while (getline(inputFile, line))
{
    cout << line << '\n';
}
```

PIECE	EXPLANATION
`string line;`	Stores one source line as a dynamically sized string.
`getline(inputFile, line)`	Extracts text through the next newline (the delimiter is not retained).
`while (...)`	Repeats while extraction is successful; eventually stops when no further line is available.
`cout << line << '\n';`	Displays the extracted line and prints a replacement newline.

Why not simply use the extraction operator?

```
// Token-oriented extraction: spaces separate items.
string word;
inputFile >> word;

// Line-oriented extraction: internal spaces remain.
string line;
getline(inputFile, line);
```

For a line such as `int number = 10;`, stream extraction into one string normally stops at the first whitespace-separated token (`int`). Line extraction keeps the indentation and spaces inside the line, although its line terminator is stripped. This is useful groundwork for preserving source layout and source locations.

Efficiency and automatic cleanup

For **N** input characters, the file-read-and-display operation is typically **O(N)** time, ignoring terminal throughput variability. The extra memory needed for the `line` variable is approximately **O(L)**, where **L** is the longest line length, rather than the whole file size. When `inputFile` goes out of scope at program exit, its destructor closes the owned file stream.

> **Important nuance**
> `getline` returning a failure state usually indicates end-of-file in this workflow, but a read failure can also occur for another reason. This sample loop is intentionally minimal; robust production error handling will later distinguish these conditions.

04 / Reproducible source and input

The following code is a representative snapshot consistent with the implemented and tested functionality after the namespace simplification. The console banner spacing may differ slightly from the local file.

src/main.cpp

```
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

int main()
{
    cout << "==============================\n";
    cout << "       CloneScope v0.1\n";
    cout << "==============================\n";
    cout << "Code Clone Analysis Tool\n";
    cout << "Status: Development Started\n";

    // Open the sample C source file
    ifstream inputFile("samples/hello.c");

    // Validate file opening before attempting to read
    if (!inputFile.is_open())
    {
        cerr << "Error: Could not open source file.\n";
        return 1;
    }

    cout << "Source file opened successfully!\n";

    string line;
    while (getline(inputFile, line))
    {
        cout << line << '\n';
    }

    return 0;
}
```

samples/hello.c (controlled test input)

```
#include <stdio.h>

int main()
{
    int number = 10;
    printf("Number: %d\n", number);
    return 0;
}
```

The sample is input **data** to the CloneScope executable; it is not compiled as part of the `g++ src/main.cpp` command. Reading the C source does not execute its `main()` or its `printf` call.

05 / Tests and observable evidence

Compilation and execution on Windows PowerShell

```
g++ -std=c++17 src/main.cpp -o clonescope.exe
.\clonescope.exe
```

The executable was run from the project root, with GCC/MSYS2 available on the PATH. A successful compile normally returns to the prompt without diagnostic output. The ignored executable is not required to be committed to Git.

Verified cases

CASE	ACTION	EXPECTED / OBSERVED
Valid source	Use `samples/hello.c`; compile and run	**Passed.** Success message and source lines printed, preserving visible spaces.
Nonexistent source	Temporarily change path to `samples/missing.c`; compile and run	**Passed.** Error: Could not open source file.
Restore sample	Restore `samples/hello.c`; recompile and run	**Passed.** Working source-line output returned.

Examples of observed terminal output

```
Source file opened successfully!
#include <stdio.h>

int main()
{
    int number = 10;
    printf("Number: %d\n", number);
    return 0;
}
```

```
// After temporarily selecting a nonexistent input:
Error: Could not open source file.
```

Additional tests not yet performed

Empty source file; only blank lines; extremely long lines; inaccessible file permissions; Unicode and non-UTF-8 encodings; Windows paths containing spaces; and source-file extensions beyond the controlled sample. These are **planned tests**, not results to claim as already verified.

Testing principle
Positive and negative tests establish baseline behavior, but two passing cases do not prove the entire file-input component is complete. The next milestones will introduce user-supplied paths, file/folder discovery, and richer diagnostics.

06 / Git history and workflow discipline

Three different operations

ACTION	WHAT IT ACTUALLY DOES	VS CODE UI
Stage	Selects changes for the next snapshot.	+ next to file in Changes.
Commit	Records the staged snapshot in local Git history.	Enter a descriptive message; click **Commit**.
Push / Publish	Sends local commits and branch updates to GitHub.	**Publish Branch** first time, then **Sync Changes** or Push.

Repository history at this checkpoint

MILESTONE	DETAIL
Foundation	Four project setup/learning commits merged to `main` through PR #1.
Working feature branch	`feat/source-file-input`, published to GitHub.
Verified first feature commit	`e55359f` - **feat: open sample C source file with ifstream**.
Next code commit (pending)	**feat: read C source file line by line** - updated main.cpp staged, not yet committed at snapshot time.
Separate documentation commit (planned)	**docs: add source-file input learning manual** - this PDF should be committed independently after the code commit.

Why two commits? The line-reading code is a tested functional change; this PDF explains and documents it. Distinct commits let an instructor review implementation changes separately from learning materials. A push can upload both commits while retaining each commit in history.

Safe GUI workflow for the next two commits

First: confirm only `src/main.cpp` is staged; enter `feat: read C source file line by line`; click Commit and verify it appears in the graph. **Second:** save this PDF into `docs/learning-notes/`; stage only the PDF; commit as `docs: add source-file input learning manual`. Finally use Sync Changes/Push and verify on GitHub.

> **What GitHub proves (and what it does not)**
> Git records code snapshots, parent relationships, author/committer information and timestamps. It demonstrates incremental changes, but does not automatically capture time spent studying or prove that every edit occurred when a commit was made. Use clear, honest commit messages and verifiable tests.

07 / Supervisor and viva question bank

1. What is implemented now?

A C++17 CLI that opens a hardcoded C source file, validates opening, reads each line via getline, and prints it. It does not yet detect clones.

2. Why use ifstream?

It is designed for reading existing files; the input source should not be modified during analysis.

3. What does is_open() verify?

Whether the stream is associated with an open file. It does not validate source-code syntax or guarantee all future reads succeed.

4. What is a file stream?

A C++ object that provides a controlled interface between a program and file input/output operations.

5. Why does the program use string?

To hold each full line, including its internal whitespace, while getline reads from the file.

6. Why is getline better than operator>> here?

getline preserves whitespace within each line; the string extraction operator separates on whitespace.

7. When does the while loop stop?

When getline can no longer successfully extract a line, typically on end-of-file.

8. Why is a valid-file test insufficient?

A missing or inaccessible file must be handled safely. Negative testing checks a failure path.

9. What does return 1 mean?

A deliberate nonzero termination status conventionally indicating an error, not an application crash.

10. Why no explicit close()?

The ifstream destructor closes the owned stream automatically when the object leaves scope; explicit close() is optional here.

11. What does using namespace std do?

It permits unqualified standard-library names. It is valid but may create naming conflicts in larger codebases.

12. Why avoid bits/stdc++.h?

It is GCC-specific, nonstandard, and obscures precise standard-library dependencies.

13. Why does samples/hello.c work?

The executable was started from the repository root, so the relative path resolves to that file.

14. Complexity of line reading?

Approximately O(N) time for N characters read and O(L) additional memory for a longest line of length L, excluding terminal/output buffering.

15. Why keep code and documentation in separate commits?

They serve different review purposes: functional behavior versus explanation and reproducibility.

16. Does this identify plagiarism?

No. CloneScope will eventually provide explainable evidence and tags for human review, not automatic accusations.

08 / Limitations, next steps, and study drills

Engineering limitations to disclose

- Input path is hardcoded as `samples/hello.c`; the user cannot select another file through the CLI yet.
- The program immediately prints each line; it has not yet built a reusable source representation or stored original line numbers.
- Only a controlled sample C file has been used so far; complex file collections and other encodings are untested.
- The current missing-file message is generic and does not report the attempted path or error category.
- No preprocessing, lexical analysis, Type-1/Type-2 clone logic, stylometry, statistics, or export logic has been implemented yet.

Approved midterm trajectory, without scope expansion

NEAR-TERM NEXT	LATER AFTER THE INPUT READER
Accept a user-supplied C/C++ file path and validate it.	Discover C/C++ files in a selected folder.
Represent source text with paths and source line numbers.	Preprocess, tokenize, and form function/block-level representations.
Test file errors and empty files.	Type-1 and Type-2 detection; tags, starter stylometry, statistics, CSV/JSON export.

Type-3 and restricted Type-4 screening, complete evidence fusion, and broad validation belong to later approved milestones, not this session.

Five self-check exercises before the next session

- Explain in one sentence what changes between file **opening** and file **reading**.
- Predict the program behavior when `samples/hello.c` is missing.
- Explain why `getline` outputs `\n` explicitly.
- Identify how running the same executable from a different current directory might affect its relative path.
- Describe why the PDF should be committed separately from the tested code.

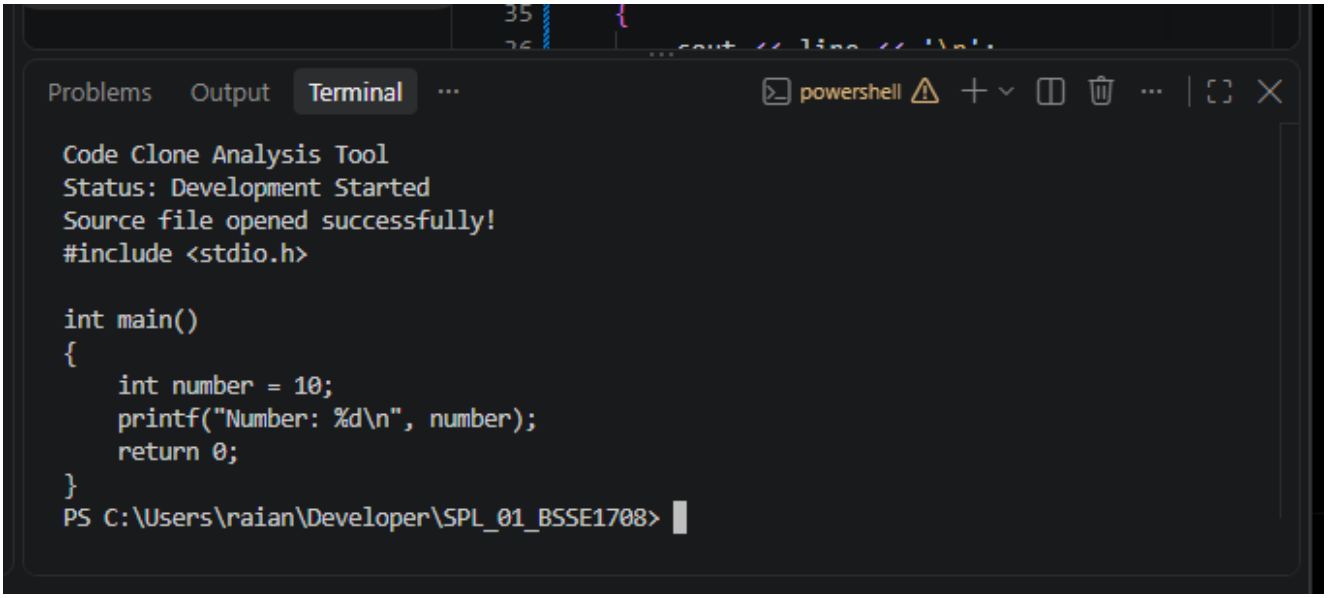
> **Suggested 30-second progress update**
> “My SPL-1 repository foundation is merged into main. On a new feature branch I implemented opening a controlled C file with ifstream, added opening-failure handling, and then extended it to line-by-line reading using getline. I verified both successful and missing-file cases. The next step is moving from a hardcoded sample path toward general local source-file input; clone detection is not implemented yet.”

Repository: github.com/raian-x8/SPL_01_BSSE1708 **Branch:** feat/source-file-input
Manual version: 1.0 **Documentation date:** 25 October 2026

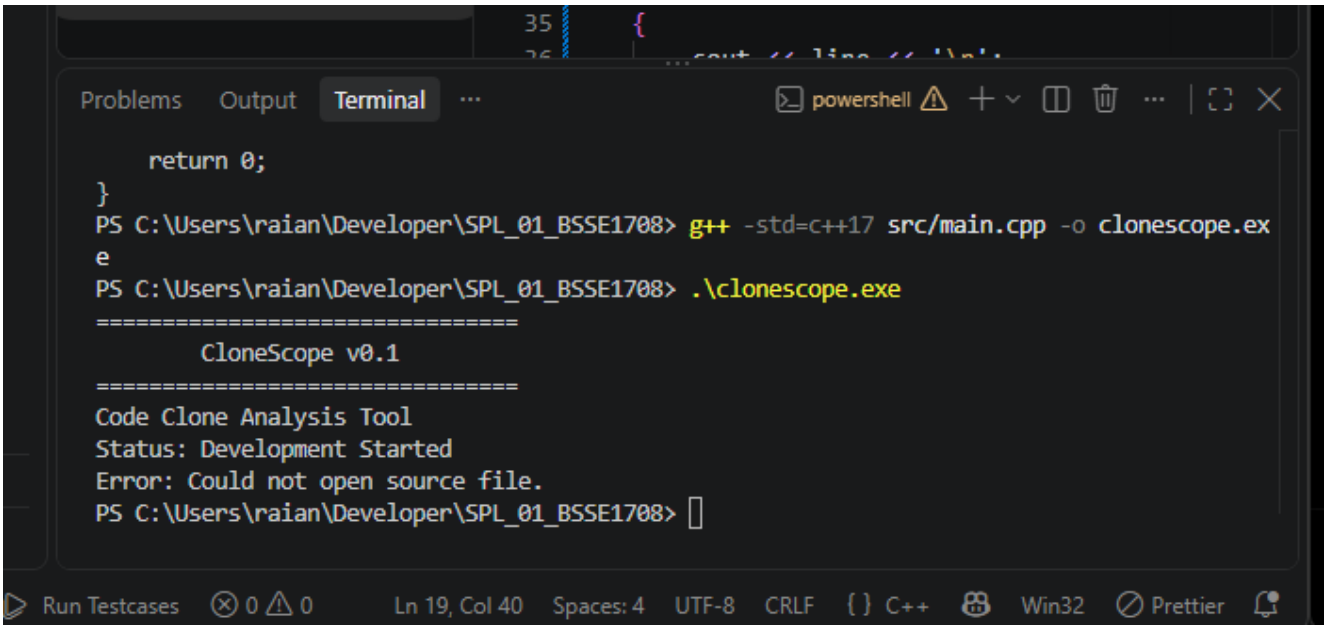
Appendix / Screenshot evidence

The following images were supplied during this live development session. They support the two observed runtime behaviors; the narrative and code examples in earlier sections explain the implementation.

A. Successful source-file reading after path restoration



B. Opening failure for a missing source file



Note: The console screenshots alone do not prove every edge case; they document the tested scenarios performed during this session.